University of Central Punjab

Faculty of Information Technology and Computer Science

Course: Operating Systems
Lab No.: GL-4
Lab Title: Inter-Process Communication (IPC) Using Ordinary (Unnamed) Pipes

1. Lab Objectives

By the end of this lab, students will be able to:

1. Understand Inter-Process Communication (IPC).
2. Understand the concept and working of ordinary pipes.
3. Create a pipe using the `pipe()` system call.
4. Use `read()` and `write()` to transfer data through a pipe.
5. Use `fork()` to create a child process.
6. Implement communication between parent and child processes.
7. Understand how to close unused pipe ends.
8. Implement one-way and two-way communication using pipes.

2. Introduction to Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism that allows two or more processes to exchange information with each other.

Normally, each process has its own memory. One process cannot directly access another process's private memory. IPC provides different techniques to exchange data between processes.

Common IPC mechanisms include:

- Pipes
- Message queues
- Shared memory
- Sockets
- Signals

In this lab, we will study **ordinary (unnamed) pipes**.

3. What Is a Pipe?

A pipe is an IPC mechanism used to transfer data from one process to another.

Think of a pipe as a water pipe:

- One end is used to put data into the pipe.
- The other end is used to take data out of the pipe.

An ordinary pipe normally transfers data in one direction. If two-way communication is required, we generally use two pipes.

A pipe is created using the `pipe()` system call in C.

3.1 Pipe File Descriptors

When we create a pipe, the operating system provides two file descriptors:

- `pfd[0]` — Read end of the pipe.
- `pfd[1]` — Write end of the pipe.

Important: The write end is used to send data, while the read end is used to receive data.

4. Important System Calls

4.1 `pipe()`

The `pipe()` system call creates a pipe.

Syntax:

```
int pfd[2];
pipe(pfd);
```

Explanation:

- `int pfd[2];` creates an array to store two file descriptors.
- `pfd[0]` stores the read-end descriptor.
- `pfd[1]` stores the write-end descriptor.
- `pipe(pfd)` asks the operating system to create the pipe.

If `pipe()` returns `-1`, pipe creation has failed.

4.2 `write()`

The `write()` system call sends data through a file descriptor.

Syntax:

```
write(pfd[1], message, size);
```

Explanation:

- `pfd[1]` specifies the write end.
- `message` contains the data.
- `size` specifies how many bytes to write.

Example:

```
char message[] = "Hello";
write(pfd[1], message, sizeof(message));
```

This writes the message, including its terminating null character, into the pipe.

4.3 `read()`

The `read()` system call receives data from a file descriptor.

Syntax:

```
read(pfd[0], buffer, size);
```

Explanation:

- `pfd[0]` specifies the read end.
- `buffer` stores the received data.
- `size` specifies the maximum number of bytes to read.

Example:

```
char buffer[100];
int n = read(pfd[0], buffer, sizeof(buffer) - 1);
```

This reads up to 99 bytes into `buffer`. For text, the program can add `'\0'` after the number of bytes actually received.

4.4 `close()`

The `close()` system call closes a file descriptor that is no longer needed.

Example:

```
close(pfd[0]);
close(pfd[1]);
```

Closing unused pipe ends is important because it prevents unnecessary resource usage and helps the reader detect when all writers have finished.

4.5 `fork()`

The `fork()` system call creates a new process called the **child process**.

The process that calls `fork()` is called the **parent process**.

Syntax:

```
pid_t pid = fork();
```

After `fork()`:

- The parent process receives the child's process ID.
- The child process receives `0`.
- If an error occurs, `fork()` returns `-1`.

The child inherits copies of the parent's open file descriptors, including the pipe descriptors. This allows the parent and child to communicate through a pipe.

5. Working of Parent–Child Communication

The general steps are:

1. Create a pipe using `pipe()`.
2. Create a child process using `fork()`.
3. Decide which process will send data and which will receive it.
4. Close the unused pipe ends in each process.
5. Use `write()` to send data.
6. Use `read()` to receive data.
7. Close the remaining file descriptors when communication is complete.

Remember: One pipe normally supports one-way communication. For two-way communication, use two pipes—one for each direction.

6. Example 1: Creating and Using a Pipe

Objective: Create a pipe and send a message through it.

Code

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pfd[2];
    char message[] = "Hello from Pipe";

    if (pipe(pfd) == -1)
    {
        printf("Pipe creation failed\n");
        return 1;
    }

    close(pfd[0]);

    write(pfd[1], message, sizeof(message));

    close(pfd[1]);

    return 0;
}
```

Explanation

1. `#include <stdio.h>` provides `printf()`.
2. `#include <unistd.h>` provides `pipe()`, `write()`, and `close()`.
3. `int pfd[2];` stores the pipe's two file descriptors.
4. `message` stores the text to send.
5. `pipe(pfd)` creates the pipe.
6. `close(pfd[0])` closes the read end because this example only writes.
7. `write()` sends the message into the pipe.
8. `close(pfd[1])` closes the write end.
9. `return 0;` ends the program successfully.

Note: This example demonstrates writing to a pipe, but it does not read or display the message. The next example demonstrates communication between two processes.

Compile and Run

Save the program as `pipe.c`.

```
gcc pipe.c -o pipe
```

```
./pipe
```

This program does not display a message because it only writes data to the pipe.

7. Example 2: Communication Between Parent and Child

Objective: Send a message from the parent process to the child process.

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    int pfd[2];
    pid_t pid;
    char message[] = "Hello Child";
    char buffer[100];

    if (pipe(pfd) == -1)
    {
        printf("Pipe creation failed\n");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid > 0)
    {
        close(pfd[0]);

        write(pfd[1], message, sizeof(message));

        close(pfd[1]);
    }
    else
    {
        close(pfd[1]);

        int n = read(pfd[0], buffer, sizeof(buffer) - 1);

        if (n > 0)
        {
            buffer[n] = '\0';
            printf("Child received: %s\n", buffer);
        }
```

```
        close(pfd[0]);
    }

    return 0;
}
```

Step-by-Step Explanation

Step 1: Create the pipe

```
int pfd[2];
pipe(pfd);
```

The pipe is created before `fork()`, so both processes inherit its file descriptors.

Step 2: Create the child process

```
pid = fork();
```

The program now has a parent process and a child process.

Step 3: Identify the parent

```
if (pid > 0)
```

This block executes in the parent process.

Step 4: Close the unused read end

```
close(pfd[0]);
```

The parent only sends data, so it closes its read end.

Step 5: Send the message

```
write(pfd[1], message, sizeof(message));
```

The parent sends `"Hello Child"` through the pipe.

Step 6: Identify the child

```
else
```

The child executes this block because `fork()` returns `0` in the child.

Step 7: Close the unused write end

```
close(pfd[1]);
```

The child only receives data, so it closes its write end.

Step 8: Read the message

```
int n = read(pfd[0], buffer, sizeof(buffer) - 1);
```

The child reads the message into `buffer`. The variable `n` stores the number of bytes received.

Step 9: Terminate the string

```
buffer[n] = '\0';
```

This adds the null character at the end of the received text so that it can be printed as a C string.

Step 10: Display the message

```
printf("Child received: %s\n", buffer);
```

The child displays the message sent by the parent.

Compile and Run

```
gcc pipe.c -o pipe
./pipe
```

Expected Output:

```
Child received: Hello Child
```

8. Example 3: Two-Way Communication Using Two Pipes

Objective: Send an integer from the parent to the child, calculate its square in the child, and return the result to the parent.

Why Do We Need Two Pipes?

The first pipe sends the integer from the parent to the child. The second pipe sends the calculated result from the child back to the parent.

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```c
int main()
{
    int p1[2], p2[2];
    pid_t pid;
    int num = 7;
    int square;

    if (pipe(p1) == -1 || pipe(p2) == -1)
    {
        printf("Pipe creation failed\n");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid > 0)
    {
        close(p1[0]);
        close(p2[1]);

        write(p1[1], &num, sizeof(num));
        close(p1[1]);

        read(p2[0], &square, sizeof(square));

        printf("Square = %d\n", square);

        close(p2[0]);
        wait(NULL);
    }
    else
    {
        close(p1[1]);
        close(p2[0]);

        read(p1[0], &num, sizeof(num));

        square = num * num;

        write(p2[1], &square, sizeof(square));

        close(p1[0]);
        close(p2[1]);
    }

    return 0;
}
```

Step-by-Step Explanation

1. `p1` is the first pipe, used to send the number from the parent to the child.
2. `p2` is the second pipe, used to return the square from the child to the parent.
3. `num = 7` stores the number to be sent.
4. `pipe(p1)` and `pipe(p2)` create two pipes.
5. `fork()` creates the child process.
6. The parent closes the read end of `p1` because it only sends the number.
7. The parent closes the write end of `p2` because it only receives the answer.
8. `write(p1[1], &num, sizeof(num))` sends the integer to the child.
9. The child closes the unused ends of both pipes.
10. `read(p1[0], &num, sizeof(num))` receives the integer.
11. `square = num * num` calculates the square. For 7, the result is 49.
12. `write(p2[1], &square, sizeof(square))` sends the result back to the parent.
13. The parent reads the result and prints it.
14. `wait(NULL)` allows the parent to wait for the child to finish.

Compile and Run

```
gcc pipe.c -o pipe
./pipe
```

Expected Output:

```
Square = 49
```

Important: In real programs, check the return values of `pipe()`, `fork()`, `read()`, and `write()` to handle errors properly.

9. Important Concepts to Remember

9.1 What Is EOF?

EOF means **End of File**. When a pipe has no more data available and all file descriptors referring to its write end have been closed, `read()` returns `0` to indicate that no more data remains.

9.2 Why Close Unused Pipe Ends?

Every process should close the pipe ends it does not use. If a process accidentally keeps a write end open, the reader may continue waiting because the operating system cannot detect that all writers have finished.

9.3 Why Use `wait()`?

`wait()` allows a parent process to wait for a child process to finish. It also helps the parent collect the child's termination status.

9.4 Difference Between One Pipe and Two Pipes

One Pipe	Two Pipes
Normally used for one-way communication	Used for two-way communication
Data travels in one direction	Data travels in both directions using separate pipes
Example: Parent sends a message to child	Example: Parent sends a number and child returns its square

10. Graded Lab Tasks

Total Marks: 15

Task 1: Parent–Child Sorting (5 Marks)

Write a C program in which:

1. The parent process receives an array of integers through command-line arguments.
2. The parent sends the array to the child process through a pipe.
3. The child sorts the array in ascending order.
4. The child sends the sorted array back to the parent using a second pipe.
5. The parent displays the sorted array.

Hint: Use two pipes, `fork()`, `read()`, `write()`, and `close()`.

Task 2: Find the Minimum (5 Marks)

Write a C program in which:

1. The parent asks the user to enter `N` integers.
2. The parent divides the numbers into two groups.
3. The parent creates two child processes.
4. Each child finds the minimum value in its assigned group.
5. Each child sends its minimum value to the parent through a pipe.
6. The parent compares both values and displays the overall minimum.

Hint: Each child should process only its assigned group.

Task 3: Parent Controls Child Processes (5 Marks)

Modify Task 2 so that:

1. Both child processes wait for a signal from the parent before starting their work.
2. The parent sends permission to both children when they can proceed.
3. Each child calculates the minimum value in its group.
4. Each child sends its result to the parent.
5. Before exiting, each child waits for permission from the parent to terminate.

Hint: Use pipes to send control messages between the parent and child processes. Ensure that the processes follow a clear communication sequence to avoid deadlocks.

11. Student Practice Questions

Students must solve the following questions independently.

Question 1: Even and Odd Numbers Using Pipes

Write a C program in which the parent process inputs five integers and sends them to the child process using a pipe. The child process should separate the numbers into even and odd numbers and send both groups back to the parent using a second pipe. The parent should display the even and odd numbers separately.

Requirements:

- Use `pipe()` and `fork()`.
- Use two pipes for two-way communication.
- Close unused pipe ends.
- Use `read()` and `write()` to transfer the data.

Question 2: Find the Largest Number Using Two Pipes

Write a C program in which the parent process inputs five integers and sends them to the child process through the first pipe. The child process finds the largest number and sends the result back to the parent through the second pipe. The parent displays the largest number.

Requirements:

- Use `pipe()` and `fork()`.
- The child must calculate the largest number.
- Use two pipes for communication in both directions.
- Close unused pipe ends and display the result in the parent process.